

CSE 3204
Formal Language, Automata and Computability



Lecture 8
**Application of Pumping Lemma and Closure Properties of
Regular Languages**

Application of Pumping Lemma

Example 4.2: Let us show that the language L_{eq} consisting of all strings with an equal number of 0's and 1's (not in any particular order) is not a regular language. In terms of the “two-player game” described in the box on “The Pumping Lemma as an Adversarial Game,” we shall be player 1 and we must deal with whatever choices player 2 makes. Suppose n is the constant that must exist if L_{eq} is regular, according to the pumping lemma; i.e., “player 2” picks n . We shall pick $w = 0^n 1^n$, that is, n 0's followed by n 1's, a string that surely is in L_{eq} .

Now, “player 2” breaks our w up into xyz . All we know is that $y \neq \epsilon$, and $|xy| \leq n$. However, that information is very useful, and we “win” as follows. Since $|xy| \leq n$, and xy comes at the front of w , we know that x and y consist only of 0's. The pumping lemma tells us that xz is in L_{eq} , if L_{eq} is regular. This conclusion is the case $k = 0$ in the pumping lemma.¹ However, xz has n 1's, since all the 1's of w are in z . But xz also has fewer than n 0's, because we lost the 0's of y . Since $y \neq \epsilon$ we know that there can be no more than $n - 1$ 0's among x and z . Thus, after assuming L_{eq} is a regular language, we have proved a fact known to be false, that xz is in L_{eq} . We have a proof by contradiction of the fact that L_{eq} is not regular. $\square$

Application of Pumping Lemma

Example 4.3: Let us show that the language L_{pr} consisting of all strings of 1's whose length is a prime is not a regular language. Suppose it were. Then there would be a constant n satisfying the conditions of the pumping lemma. Consider some prime $p \geq n + 2$; there must be such a p , since there are an infinity of primes. Let $w = 1^p$.

By the pumping lemma, we can break $w = xyz$ such that $y \neq \epsilon$ and $|xy| \leq n$. Let $|y| = m$. Then $|xz| = p - m$. Now consider the string $xy^{p-m}z$, which must be in L_{pr} by the pumping lemma, if L_{pr} really is regular. However,

$$|xy^{p-m}z| = |xz| + (p - m)|y| = p - m + (p - m)m = (m + 1)(p - m)$$

It looks like $|xy^{p-m}z|$ is not a prime, since it has two factors $m + 1$ and $p - m$. However, we must check that neither of these factors are 1, since then $(m + 1)(p - m)$ might be a prime after all. But $m + 1 > 1$, since $y \neq \epsilon$ tells us $m \geq 1$. Also, $p - m > 1$, since $p \geq n + 2$ was chosen, and $m \leq n$ since

$$m = |y| \leq |xy| \leq n$$

Thus, $p - m \geq 2$.

Again we have started by assuming the language in question was regular, and we derived a contradiction by showing that some string not in the language was required by the pumping lemma to be in the language. Thus, we conclude that L_{pr} is not a regular language. $\square$

Closure Properties of Regular Languages

1. The union of two regular languages is regular.
2. The intersection of two regular languages is regular.
3. The complement of a regular language is regular.
4. The difference of two regular languages is regular.
5. The reversal of a regular language is regular.
6. The closure (star) of a regular language is regular.
7. The concatenation of regular languages is regular.
8. A homomorphism (substitution of strings for symbols) of a regular language is regular.
9. The inverse homomorphism of a regular language is regular.

Closure of Regular Languages Under Boolean Operations

1. Let L and M be languages over alphabet Σ . Then $L \cup M$ is the language that contains all strings that are in either or both of L and M .

Closure Under Union

Theorem 4.4: If L and M are regular languages, then so is $L \cup M$.

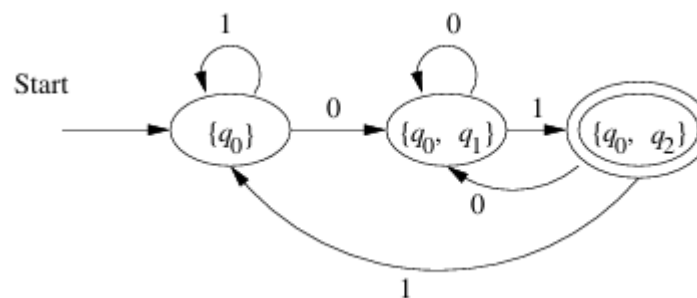
PROOF: This proof is simple. Since L and M are regular, they have regular expressions; say $L = L(R)$ and $M = L(S)$. Then $L \cup M = L(R + S)$ by the definition of the $+$ operator for regular expressions. $\square$

Closure Under Complementation

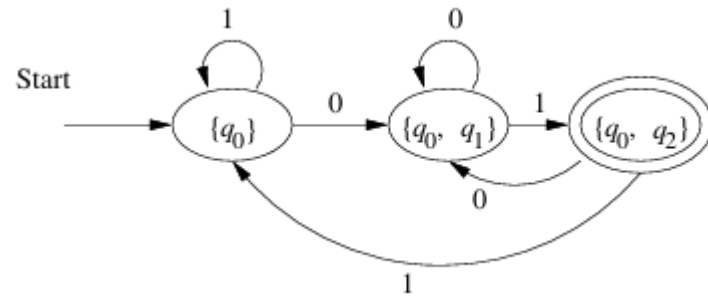
1. Convert the regular expression to an ϵ -NFA.
2. Convert that ϵ -NFA to a DFA by the subset construction.
3. Complement the accepting states of that DFA.
4. Turn the complement DFA back into a regular expression using the construction of Sections 3.2.1 or 3.2.2.

Theorem 4.5: If L is a regular language over alphabet Σ , then $\bar{L} = \Sigma^* - L$ is also a regular language.

PROOF: Let $L = L(A)$ for some DFA $A = (Q, \Sigma, \delta, q_0, F)$. Then $\bar{L} = L(B)$, where B is the DFA $(Q, \Sigma, \delta, q_0, Q - F)$. That is, B is exactly like A , but the accepting states of A have become nonaccepting states of B , and vice versa. Then w is in $L(B)$ if and only if $\hat{\delta}(q_0, w)$ is in $Q - F$, which occurs if and only if w is not in $L(A)$. $\square$



Closure of Regular Languages Under Boolean Operations



Regular expression: $(0+1)^*01$

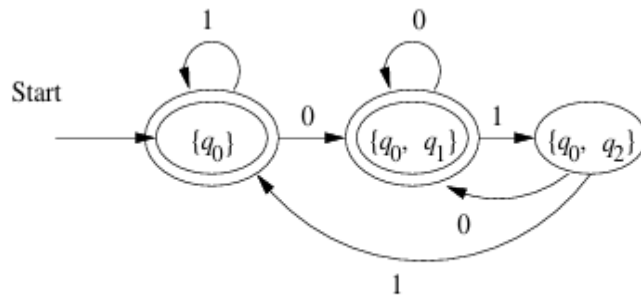


Figure 4.2: DFA accepting the complement of the language $(0 + 1)^*01$

The language L_{eq} consisting of strings with an equal number of 0s and 1s is not regular.

The language M consisting of strings with unequal number of 0s and 1s.

M is complement of L_{eq} .

As L_{eq} is not regular, M is not regular.

Closure under Intersection

$$L \cap M = \overline{\overline{L} \cup \overline{M}}$$

Closure under Intersection

Theorem 4.8: If L and M are regular languages, then so is $L \cap M$.

PROOF: Let L and M be the languages of automata $A_L = (Q_L, \Sigma, \delta_L, q_L, F_L)$ and $A_M = (Q_M, \Sigma, \delta_M, q_M, F_M)$. Notice that we are assuming that the alpha-

The remaining details are simple. The start state of A is the pair of start states of A_L and A_M . Since we want to accept if and only if both automata accept, we select as the accepting states of A all those pairs (p, q) such that p is an accepting state of A_L and q is an accepting state of A_M . Formally, we define:

$$A = (Q_L \times Q_M, \Sigma, \delta, (q_L, q_M), F_L \times F_M)$$

where $\delta((p, q), a) = (\delta_L(p, a), \delta_M(q, a))$.

To see why $L(A) = L(A_L) \cap L(A_M)$, first observe that an easy induction on $|w|$ proves that $\hat{\delta}((q_L, q_M), w) = (\hat{\delta}_L(q_L, w), \hat{\delta}_M(q_M, w))$. But A accepts w if and only if $\hat{\delta}((q_L, q_M), w)$ is a pair of accepting states. That is, $\hat{\delta}_L(q_L, w)$ must be in F_L , and $\hat{\delta}_M(q_M, w)$ must be in F_M . Put another way, w is accepted by A if and only if both A_L and A_M accept w . Thus, A accepts the intersection of

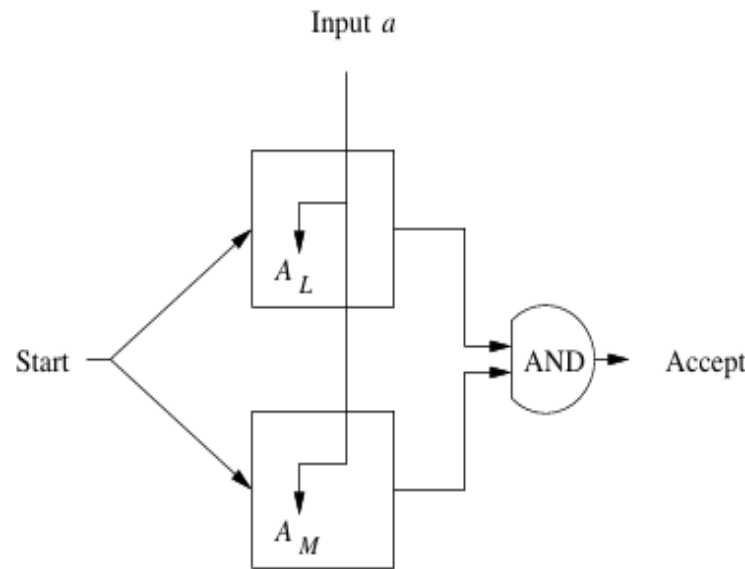
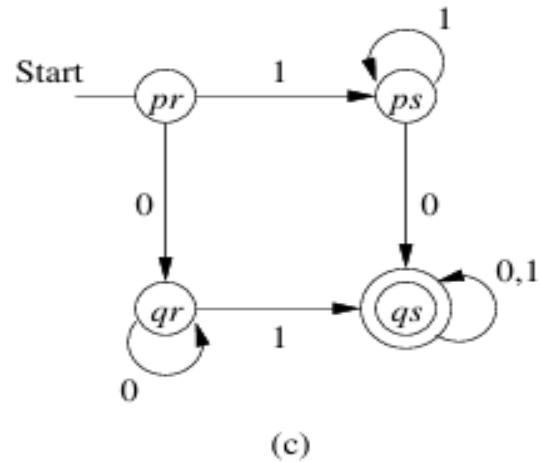
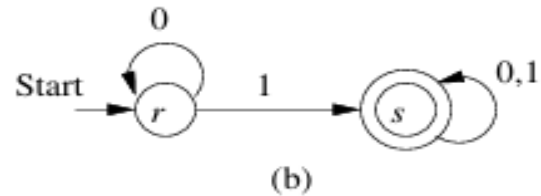
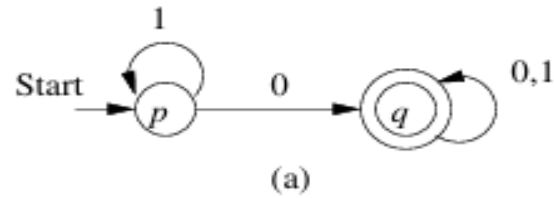


Figure 4.3: An automaton simulating two other automata and accepting if and only if both accept

Closure under Intersection



It is easy to argue that this automaton accepts the intersection of the first two languages: those strings that have both a 0 and a 1. State pr represents only the initial condition, in which we have seen neither 0 nor 1. State qr means that we have seen only 0's, while state ps represents the condition that we have seen only 1's. The accepting state qs represents the condition where we have seen both 0's and 1's. $\square$

Figure 4.4: The product construction

Reversal:

The *reversal* of a string $a_1a_2\cdots a_n$ is the string written backwards, that is, $a_na_{n-1}\cdots a_1$. We use w^R for the reversal of string w . Thus, 0010^R is 0100 , and $\epsilon^R = \epsilon$.

The reversal of a language L , written L^R , is the language consisting of the reversals of all its strings. For instance, if $L = \{001, 10, 111\}$, then $L^R = \{100, 01, 111\}$.

Proof by FA:

Given a language L that is $L(A)$ for some finite automaton, perhaps with nondeterminism and ϵ -transitions, we may construct an automaton for L^R by:

1. Reverse all the arcs in the transition diagram for A .
2. Make the start state of A be the only accepting state for the new automaton.
3. Create a new start state p_0 with transitions on ϵ to all the accepting states of A .

The result is an automaton that simulates A “in reverse,” and therefore accepts a string w if and only if A accepts w^R . Now, we prove the reversal theorem formally.

Theorem 4.11: If L is a regular language, so is L^R .

PROOF: Assume L is defined by regular expression E . The proof is a structural induction on the size of E . We show that there is another regular expression E^R such that $L(E^R) = (L(E))^R$; that is, the language of E^R is the reversal of the language of E .

BASIS: If E is ϵ , $\emptyset$, or $\mathbf{a}$, for some symbol a , then E^R is the same as E . That is, we know $\{\epsilon\}^R = \{\epsilon\}$, $\emptyset^R = \emptyset$, and $\{a\}^R = \{a\}$.

INDUCTION: There are three cases, depending on the form of E .

1. $E = E_1 + E_2$. Then $E^R = E_1^R + E_2^R$. The justification is that the reversal of the union of two languages is obtained by computing the reversals of the two languages and taking the union of those languages.
2. $E = E_1E_2$. Then $E^R = E_2^RE_1^R$. Note that we reverse the order of the two languages, as well as reversing the languages themselves. For instance, if $L(E_1) = \{01, 111\}$ and $L(E_2) = \{00, 10\}$, then $L(E_1E_2) = \{0100, 0110, 11100, 11110\}$. The reversal of the latter language is

$$\{0010, 0110, 00111, 01111\}$$

If we concatenate the reversals of $L(E_2)$ and $L(E_1)$ in that order, we get

$$\{00, 01\}\{10, 111\} = \{0010, 00111, 0110, 01111\}$$

which is the same language as $(L(E_1E_2))^R$. In general, if a word w in $L(E)$ is the concatenation of w_1 from $L(E_1)$ and w_2 from $L(E_2)$, then $w^R = w_2^Rw_1^R$.

Continuation of Theorem

3. $E = E_1^*$. Then $E^R = (E_1^R)^*$. The justification is that any string w in $L(E)$ can be written as $w_1w_2 \cdots w_n$, where each w_i is in $L(E)$. But

$$w^R = w_n^R w_{n-1}^R \cdots w_1^R$$

Each w_i^R is in $L(E^R)$, so w^R is in $L((E_1^R)^*)$. Conversely, any string in $L((E_1^R)^*)$ is of the form $w_1w_2 \cdots w_n$, where each w_i is the reversal of a string in $L(E_1)$. The reversal of this string, $w_n^R w_{n-1}^R \cdots w_1^R$, is therefore a string in $L(E_1^*)$, which is $L(E)$. We have thus shown that a string is in $L(E)$ if and only if its reversal is in $L((E_1^R)^*)$.

Closure Under Difference

Theorem 4.10: If L and M are regular languages, then so is $L - M$.

PROOF: Observe that $L - M = L \cap \overline{M}$. By Theorem 4.5, $\overline{M}$ is regular, and by Theorem 4.8 $L \cap \overline{M}$ is regular. Therefore $L - M$ is regular. $\square$

- Section 4.1 of Chapter 4
- Section 4.2 of Chapter 4